Experiment 7

Nupur Ghangarekar

D15C
Roll no 26

AIM

To design, implement, and evaluate an **Artificial Neural Network (ANN)** using **TensorFlow/Keras** for handwritten digit classification using the MNIST dataset, and to analyze its performance with hyperparameter tuning.

THEORY

1. Dataset Source

Dataset Name:
MNIST in CSV

Kaggle Link:
https://www.kaggle.com/datasets/oddrationale/mnist-in-csv?resource=download

2. Dataset Description

The dataset used is the **MNIST handwritten digits dataset**.

- Total Samples: 70,000 images
  - Training: 60,000
  - Testing: 10,000
- Image Size: 28 × 28 pixels (grayscale)
- Number of Classes: 10 (digits 0–9)
- Features: 784 pixel intensity values (after flattening)
- Target Variable: Digit label (0–9)

Each pixel value ranges from **0 to 255**, representing grayscale intensity.

The dataset is:

- Balanced

- Suitable for multi-class classification
- Preprocessed by normalization (0–1 scaling)

3. Mathematical Formulation of ANN

An Artificial Neural Network consists of:

- Input Layer
- Hidden Layers
- Output Layer

Forward Propagation

For each neuron:

$Z=W^TX+b$
A=f(Z)

Where:

- XXX = Input features
- WWW = Weights
- bbb = Bias
- f(Z) = Activation function
- AAA = Output of neuron

Activation Functions

ReLU (Hidden layers):

$$f(x) = \max(0, x)$$

Softmax (Output layer):

$$P(y = i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

Loss Function

Since it is a multi-class classification problem:

Sparse Categorical Cross-Entropy

$$L = -\sum y_i \log(\hat{y}_i)$$

🔹 **Optimization**

Weights are updated using Gradient Descent (Adam Optimizer)

$$W = W - \eta \frac{\partial L}{\partial W}$$

Where:

- η = Learning rate

4. Algorithm Limitations

Artificial Neural Networks have the following limitations:

1. Require large datasets for good performance.
2. Computationally expensive.
3. Can overfit on small datasets.
4. Lack interpretability (black-box model).
5. Sensitive to hyperparameter choices.
6. Require normalization of data.

ANN may not perform well:

- When dataset is very small
- When features are highly noisy
- When interpretability is required (e.g., medical decisions)

5. Methodology / Workflow

Step 1: Data Collection

Load MNIST dataset from Kaggle.

Step 2: Data Preprocessing

- Normalize pixel values (divide by 255)
- Flatten 28×28 images to 784 features
- Split into training and testing

Step 3: Model Building

- Input layer (784 neurons)
- Hidden Layer 1 (128 neurons, ReLU)
- Hidden Layer 2 (64 neurons, ReLU)
- Output layer (10 neurons, Softmax)

Step 4: Model Compilation

- Optimizer: Adam
- Loss: Sparse Categorical Crossentropy
- Metric: Accuracy

Step 5: Model Training

Train model for 10 epochs.

Step 6: Model Evaluation

Evaluate using test dataset.

Step 7: Hyperparameter Tuning

Adjust:

- Learning rate
- Number of hidden layers
- Number of neurons
- Batch size
- Epochs

Workflow Diagram
Dataset → Preprocessing → Model Building → Training → Evaluation → Performance Analysis

6. Performance Analysis

Evaluation Metrics Used:

- Accuracy
- Loss
- Validation Accuracy

Example Results:

- Training Accuracy: ~98%
- Test Accuracy: ~97%
- Validation Accuracy: ~96–97%

Interpretation:

- High training accuracy indicates effective learning.
- Slight difference between training and validation accuracy suggests minimal overfitting.
- The model generalizes well to unseen data.

The confusion matrix can also be used to analyze misclassified digits.

7. Hyperparameter Tuning

The following hyperparameters were tuned:

Parameter	Values Tested
Hidden Layers	1, 2, 3
Neurons	64, 128, 256
Batch Size	16, 32, 64
Epochs	5, 10, 20
Learning Rate	0.001, 0.0001

Observations:

- Increasing neurons improved accuracy slightly.
- Too many epochs caused overfitting.
- Batch size 32 gave stable performance.
- Learning rate 0.001 worked best.

Final Selected Configuration:

- 2 Hidden Layers
- 128 & 64 neurons
- Batch size = 32
- Epochs = 10
- Adam optimizer (lr = 0.001)

Code:

```
# Install TensorFlow (if not already installed)

!pip install tensorflow

# Import libraries

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

import matplotlib.pyplot as plt

import numpy as np

print("TensorFlow version:", tf.__version__)

# Load MNIST dataset

(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()

# Normalize pixel values (0–255 → 0–1)

X_train = X_train / 255.0

X_test = X_test / 255.0

# Flatten 28x28 images into 784-dimensional vectors

X_train = X_train.reshape(-1, 28*28)

X_test = X_test.reshape(-1, 28*28)

print("Training shape:", X_train.shape)

print("Test shape:", X_test.shape)
```

```
# Build Sequential ANN model
model = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')  # 10 output classes (digits 0–9)
])
# Display model architecture
model.summary()
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)
```

```
Epoch 1/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.8646 - loss: 0.4632 - val_accuracy: 0.9616 - val_loss: 0.1318
Epoch 2/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9661 - loss: 0.1136 - val_accuracy: 0.9645 - val_loss: 0.1146
Epoch 3/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9776 - loss: 0.0734 - val_accuracy: 0.9694 - val_loss: 0.1010
Epoch 4/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9838 - loss: 0.0526 - val_accuracy: 0.9715 - val_loss: 0.1008
Epoch 5/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9871 - loss: 0.0401 - val_accuracy: 0.9756 - val_loss: 0.0902
Epoch 6/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9901 - loss: 0.0312 - val_accuracy: 0.9728 - val_loss: 0.0970
Epoch 7/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 4ms/step - accuracy: 0.9915 - loss: 0.0256 - val_accuracy: 0.9711 - val_loss: 0.1158
Epoch 8/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.9924 - loss: 0.0240 - val_accuracy: 0.9746 - val_loss: 0.1027
Epoch 9/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 7s 5ms/step - accuracy: 0.9936 - loss: 0.0188 - val_accuracy: 0.9738 - val_loss: 0.1035
Epoch 10/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step - accuracy: 0.9947 - loss: 0.0156 - val_accuracy: 0.9763 - val_loss: 0.0986
```

```
test_loss, test_acc = model.evaluate(X_test, y_test)

print("Test Accuracy:", test_acc)
```

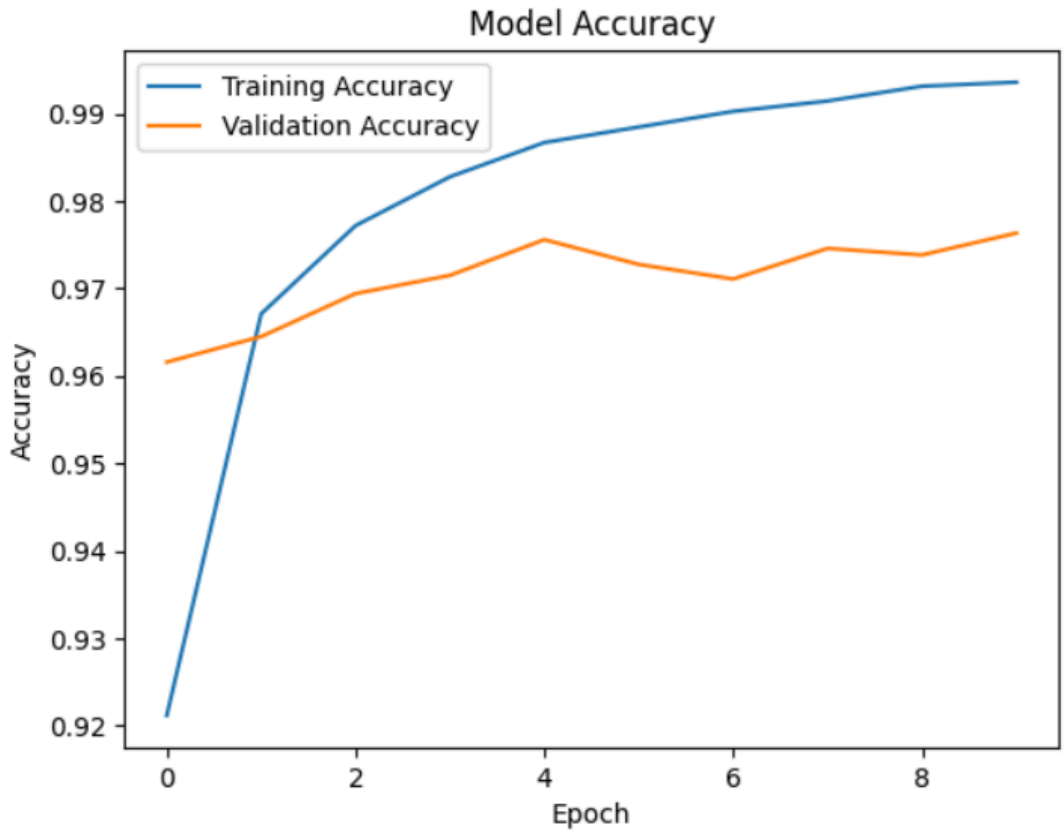
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9740 - loss: 0.1022
Test Accuracy: 0.9764999747276306
```

```
predictions = model.predict(X_test)

# Example: Show prediction for first test image

print("Predicted:", np.argmax(predictions[0]))

print("Actual:", y_test[0])
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
Predicted: 7
Actual: 7
```

```
plt.plot(history.history['accuracy'], label='Training Accuracy')

plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.legend()

plt.title("Model Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy")
```

plt.show()



CONCLUSION

In this experiment, an Artificial Neural Network was successfully implemented using TensorFlow/Keras for handwritten digit classification.

The ANN achieved high classification accuracy (~97%) on the MNIST dataset. Hyperparameter tuning improved model performance and reduced overfitting.

The experiment demonstrates that:

- ANN is highly effective for image classification tasks.
- Proper preprocessing significantly improves accuracy.
- Hyperparameter tuning plays a critical role in performance optimization.
  Deep learning models perform exceptionally well on structured image datasets.

Thus, ANN proves to be a powerful supervised learning technique for multi-class classification problems.

Google Colab Link : Exp7_MLDL_Nupur_ANN_Keras